DOCUMENTAZIONE TECNICA E FUNZIONALE

# StudyCoach

Una piattaforma web per studiare materiali universitari con quiz, tutor socratico e strumenti di ripasso.

**APPROCCIO BOTTOM UP**

La lettura parte dalle strutture dati e dai servizi elementari, mostra come cooperano nell'architettura e arriva ai requisiti, ai casi d'uso e alle funzioni visibili allo studente.

Edizione basata sul codice del progetto - 16 ottobre 2026

Stack principale: Python 3.13, FastAPI, Jinja2, SQLite, ChromaDB e servizi LLM configurabili.

# 01 Il progetto in sintesi

StudyCoach è un'applicazione web per organizzare esami e materiali di studio, generare esercizi fondati sui PDF caricati e guidare lo studente in un'interrogazione attiva. L'interfaccia browser e l'API sono servite dallo stesso processo FastAPI.

> **OBIETTIVO**
>
> Trasformare dispense e slide in un percorso di studio: fonti indicizzate, domande verificabili, risposta iniziale, domanda guida, rivalutazione, riepilogo e ripasso.

Il sistema integra tre tipi di contenuto: dati strutturati dello studente e delle sessioni, documenti PDF e brani indicizzati per la ricerca semantica. I modelli linguistici intervengono nei passaggi di generazione e valutazione, ricevendo il contesto ricavato dalle fonti.

## Come leggere l'architettura

- **Livello 1 - dati:** account, esami, materiali, sessioni e conversazioni.
- **Livello 2 - elaborazione:** estrazione PDF, OCR, segmentazione, embedding e ricerca.
- **Livello 3 - servizi:** quiz, tutor, chat dei dubbi, appunti e report.
- **Livello 4 - prodotto:** API HTTP, pagine web e casi d'uso dello studente.

# 02 Fondazioni: dati e persistenza

Il database SQLite conserva le entità applicative. I file PDF sono salvati sul filesystem; ChromaDB conserva i brani indicizzati con metadati utili a ricerca e citazioni.

| Entità | Informazioni principali | Relazione |
|---|---|---|
| Studente | Identità, hash della password, sessione di accesso | Possiede esami e chat |
| Esame | Nome e data di creazione | Raggruppa materiali e sessioni |
| Materiale | Nome, percorso PDF, impronta SHA-256, pagine e chunk | Appartiene a un esame |
| Sessione | Quiz, tentativi, turno attivo, durata, stato, voto e report | Appartiene a esame e studente |
| Chat / dubbio | Thread, domanda guida, risposta e risultato | Associati a studente ed esame |
| Tentativo di accesso | Contatori e finestra temporale dei login | Protegge l'autenticazione |

La sessione persiste il quiz e il turno corrente in JSON. Il registro dei tentativi contiene le risposte e i voti delle domande. La durata attiva è accumulata nel database, così pausa e ripresa conservano il tempo di studio.

## Il legame tra database e indice vettoriale

Ogni materiale ha un'impronta del file e una riga nel database. I chunk di Chroma conservano origine e pagina; la collezione è distinta per studente ed esame. La selezione di fonti usa gli identificativi dei materiali, ripresi anche dal quiz della sessione.

# 03 Dal PDF ai brani consultabili

L'ingestione prepara il contenuto su cui si basano quiz, tutor e appunti.

| Passo | Elaborazione | Risultato |
|---|---|---|
| 1. Caricamento | Il server riceve il PDF e registra il file | File associato all'esame |
| 2. Lettura | Il testo viene estratto per pagina; l'OCR tratta le pagine senza testo | Testo con numero di pagina |
| 3. Segmentazione | Il testo è diviso in brani con sovrapposizione e limite del tokenizer | Chunk leggibili dal modello |
| 4. Indicizzazione | Embedding locale e salvataggio in Chroma con metadati | Collezione ricercabile |
| 5. Recupero | Ricerca semantica filtrabile per materiali selezionati | Brani con fonte e pagina |

## Ricerca aumentata da fonti

Il flusso RAG (retrieval augmented generation) recupera brani pertinenti dall'indice e li passa ai servizi di generazione. I metadati della fonte accompagnano i brani e permettono di mostrare riferimenti alla pagina. Per i quiz, la citazione proposta dal modello viene confrontata con il testo indicizzato.

> **TRACCIABILITÀ DELLE FONTI**
>
> Lo studente può usare tutti i materiali di un esame oppure selezionarne alcuni. La scelta viene conservata nella sessione e applicata alla ricerca usata dal tutor.

# 04 Servizi applicativi

I moduli del backend trasformano i brani e gli eventi della sessione in attività di apprendimento.

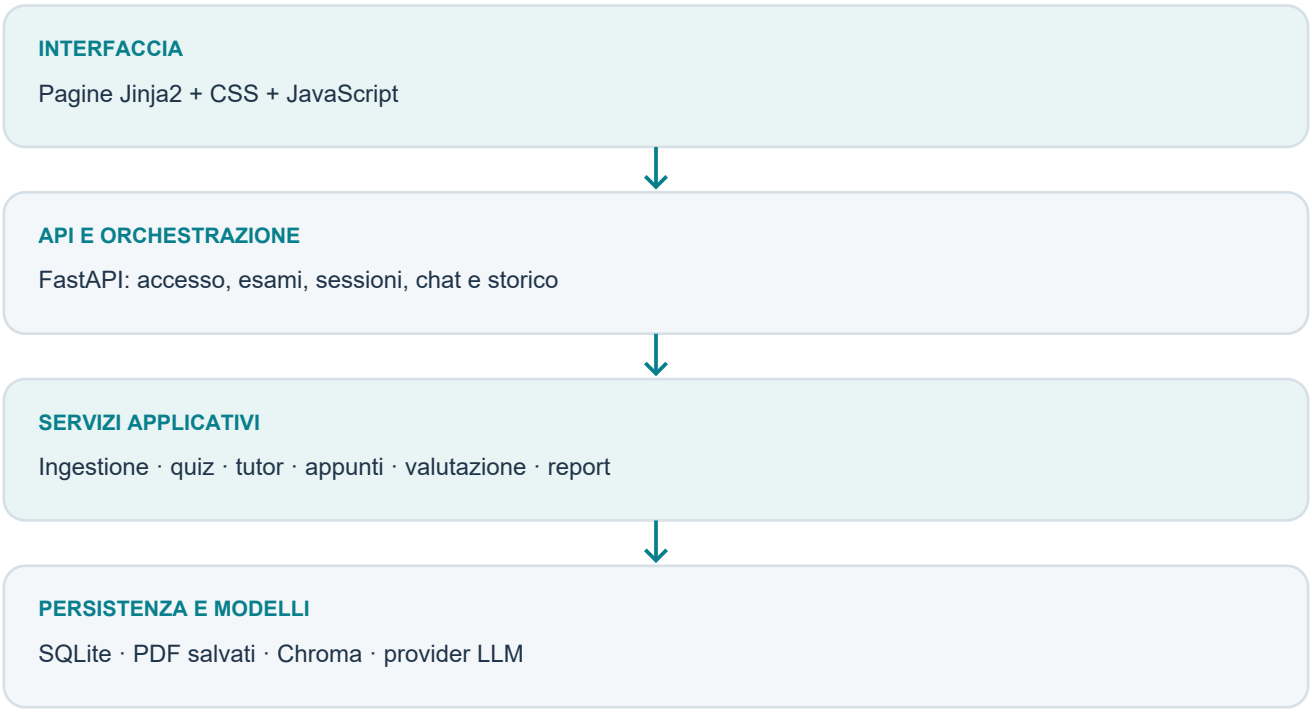
| Servizio | Responsabilità |
|---|---|
| QuizGenerator | Compone contesti dalle fonti, genera domande con difficoltà e argomento, verifica la presenza delle citazioni. |
| SocraticTutor | Gestisce il turno: prima risposta, domanda guida, seconda risposta, valutazione, contestazione e chiusura. |
| AdaptiveEngine | Ricostruisce i risultati dai tentativi e privilegia gli argomenti deboli nella scelta della prossima domanda. |
| NotesGenerator | Produzione di appunti dai materiali selezionati o dallo storico della sessione, con output Markdown e PDF. |
| ReportGenerator | Formatta e salva media, argomenti e sintesi in report Markdown e PDF. |
| DoubtTutor | Apre una conversazione su un dubbio, pone una domanda guida e restituisce il feedback alla risposta. |

## Valutazione e modelli

Le risposte dei modelli sono interpretate come dati strutturati. Il tutor usa controlli di coerenza tra voto, punti corretti, elementi mancanti e motivazione; il riesame della contestazione verifica la relazione tra voto iniziale, variazione e voto finale. La scelta del provider è distinta per tutor, quiz e appunti tramite configurazione.

# 05 Architettura complessiva

L'applicazione è organizzata in livelli: interfaccia, API, servizi di dominio e persistenza. Il server FastAPI coordina tutti i flussi HTTP e inizializza il database all'avvio.




L'interfaccia usa template Jinja2 e file statici per pagine, timer e aggiornamenti di avanzamento. L'API espone le stesse operazioni di dominio a client HTTP. I router separano autenticazione, esami, sessioni e operazioni aggiuntive.

L'accesso alle risorse è collegato allo studente autenticato. Le sessioni browser usano un cookie HttpOnly; i percorsi API controllano l'appartenenza di esami e sessioni. Le risposte HTTP includono un identificativo di richiesta per correlare gli eventi di esecuzione.

# 06 Requisiti iniziali

I requisiti descrivono il comportamento richiesto a un assistente di studio basato sui materiali dello studente.

| ID | Requisito funzionale |
|---|---|
| RF01 | Registrare lo studente e consentire l'accesso personale. |
| RF02 | Creare esami e associare più PDF allo stesso esame. |
| RF03 | Estrarre, indicizzare e recuperare contenuti dai PDF con fonte e pagina. |
| RF04 | Generare quiz dalle fonti selezionate e conservare le domande nella sessione. |
| RF05 | Guidare la risposta in due passaggi, valutarla e permettere una contestazione. |
| RF06 | Adattare l'ordine delle domande ai risultati ottenuti. |
| RF07 | Produrre appunti, cronologia e report di studio. |
| RF08 | Gestire dubbi in chat collegate agli esami. |

## Requisiti trasversali

- **Persistenza:** conservare esami, fonti, progressi, tentativi, durata e report.
- **Separazione dei dati:** associare ogni risorsa allo studente proprietario.
- **Usabilità:** offrire pagine web e avanzamento delle operazioni più lunghe.
- **Configurabilità:** scegliere modelli e parametri operativi dall'ambiente e da config/app.toml.

# 07 Casi d'uso principali

L'attore primario è lo studente autenticato. Ogni caso d'uso combina i componenti descritti nelle pagine precedenti.

| Caso | Azione dello studente | Esito |
|---|---|---|
| UC01 | Crea un account e accede | Area personale disponibile |
| UC02 | Crea o rinomina un esame | Contenitore per fonti e sessioni |
| UC03 | Carica e seleziona uno o più PDF | Materiali indicizzati e consultabili |
| UC04 | Avvia una sessione con 3-10 domande dall'interfaccia | Quiz persistito e primo turno attivo |
| UC05 | Risponde, riflette sulla domanda guida e risponde di nuovo | Feedback, voto e prossima domanda |
| UC06 | Contesta un voto | Riesame motivato e voto finale |
| UC07 | Genera appunti o apre una chat su un dubbio | Materiale di ripasso e conversazione salvata |
| UC08 | Consulta cronologia e report; riprende una sessione | Progressi, durata, media e PDF del report |

## Percorso tipico

**DALLE FONTI AL RIPASSO**

Registrazione → esame → PDF → scelta delle fonti → quiz → tutor a due risposte → eventuale contestazione → report → appunti e cronologia.

# 08 Funzioni implementate e uso operativo

Le funzioni sono accessibili dalla UI servita da FastAPI; gli endpoint sono documentati automaticamente in **/docs**.

| Area | Funzioni disponibili |
| --- | --- |
| Account | Registrazione, login, logout, cookie di sessione, controllo dei tentativi di accesso. |
| Esami e fonti | Creazione, rinomina, eliminazione, caricamento PDF, verifica, reindicizzazione e selezione delle fonti. |
| Studio | Quiz, tutor socratico, valutazione, contestazione, scelta adattiva, timer, interruzione e ripresa. |
| Ripasso | Appunti dai PDF in PDF/Markdown, appunti dalla sessione, chat dei dubbi e storico conversazioni. |
| Risultati | Storico sessioni, media, argomenti, report PDF e rigenerazione del report. |

## Avvio locale

Requisiti: Python 3.13 o successivo, **uv** e le credenziali del provider scelto nel file **.env**. Dalla cartella del progetto: **uv sync**; quindi **uv run uvicorn backend.api.main:app --reload --host 127.0.0.1 --port 8000**. L'interfaccia è disponibile su **http://127.0.0.1:8000/**.

## Mappa dei moduli

**backend/db/models.py** definisce i dati; **backend/agents/** contiene ingestione, generazione, tutor e report; **backend/api/** espone i router; **frontend/web.py** e **frontend/templates/** realizzano le pagine; **config/app.toml** contiene le opzioni non segrete.

> **RISULTATO**
>
> StudyCoach unisce gestione dei materiali, interrogazione guidata e storico dei risultati in un unico ambiente di studio accessibile dal browser.